

Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Сибирский государственный университет телекоммуникаций и
информатики»
(СибГУТИ)

Кафедра ТСиВС

ОТЧЕТ ПО КУРСОВОЙ РАБОТЕ

**По дисциплине: «Интернет-технологии и сетевое
программное обеспечение»**

**Тема: «Разработка финального проекта: Система управления
счетами (invoices-s13)»**

Студент группы ИКС-433

ФИО студента Криволапов Н. А.

Проверил:

Должность ст. преподаватель

ФИО Лошкарев А. А.

Новосибирск 2026 г.

Содержание

ВВЕДЕНИЕ	2
1 ПОСТАНОВКА ЗАДАЧИ (INVOICES-S13)	3
1.1 Назначение информационной системы	3
1.2 Функциональные требования к системе	3
1.3 Нефункциональные требования	3
1.4 Спецификация варианта s13	3
2 АРХИТЕКТУРА СИСТЕМЫ УПРАВЛЕНИЯ СЧЕТАМИ	4
2.1 Компонентная структура и распределение ролей	4
2.2 Стек протоколов и обоснование выбора	4
2.3 Модель управления данными и жизненный цикл	5
2.4 Оркестрация и Service Discovery	5
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КОМПОНЕНТОВ	6
3.1 Реализация gRPC-контракта и сервера	6
3.2 Реализация REST-шлюза на FastAPI	6
3.3 Система обработки исключительных ситуаций	6
4 КОНТЕЙНЕРИЗАЦИЯ, ТЕСТИРОВАНИЕ И РАЗВЕРТЫВАНИЕ	7
4.1 Спецификация контейнеров (Docker)	7
4.2 Сценарии верификации работоспособности	7
4.3 Автоматизация сборки и тестирования	7
4.4 Ограничения и потенциал развития	7
ЗАКЛЮЧЕНИЕ	8

ВВЕДЕНИЕ

Современные распределенные системы часто строятся по принципу разделения ответственности между независимыми компонентами. Это позволяет эффективно масштабировать отдельные части системы, использовать наиболее подходящие технологии для каждой задачи и обеспечивать высокую отказоустойчивость.

В рамках данной работы разработана учебная распределенная система управления счетами `invoices-s13` на языке Python. Система позволяет регистрировать финансовые документы, получать информацию об их текущем статусе и суммах, а также управлять списком доступных документов через единый интерфейс. Разделение обязанностей между gRPC-сервисом бэкенда и REST-шлюзом обеспечивает гибкость API и высокую производительность внутреннего взаимодействия.

1 ПОСТАНОВКА ЗАДАЧИ (INVOICES-S13)

1.1 Назначение информационной системы

Информационная система `invoices-s13` разработана для автоматизации учета финансовых документов (счетов) в рамках микросервисной архитектуры. Основная цель системы --- обеспечить надежное хранение данных о счетах и предоставить гибкий интерфейс для их создания и получения внешними потребителями.

1.2 Функциональные требования к системе

Система должна реализовывать следующий набор функций:

- Создание записи о счете с обязательным указанием суммы (`amount`).
- Получение подробной информации по идентификатору счета.
- Формирование списка всех существующих счетов через потоковую передачу данных.
- Корректная трансляция данных между различными форматами представления (Protobuf и JSON).

1.3 Нефункциональные требования

- **Стек технологий:** Использование Python для всех компонентов системы.
- **Взаимодействие:** Применение gRPC для внутренних вызовов и REST для внешних.
- **Изоляция:** Каждый компонент должен функционировать в собственном Docker-контейнере.

1.4 Спецификация варианта s13

Согласно заданию, система работает с объектом `invoice`. Основной числовой атрибут --- `amount` (тип `float`). Все компоненты системы используют идентификатор `s13` в названиях и конфигурациях.

2 АРХИТЕКТУРА СИСТЕМЫ УПРАВЛЕНИЯ СЧЕТАМИ

2.1 Компонентная структура и распределение ролей

Архитектура информационной системы `invoices-s13` базируется на паттерне <<API Gateway>>, который является стандартом для современных микросервисных приложений. Система разделена на два функциональных контура:

1. **API Gateway (Внешний контур):** Выступает в роли единой точки входа (Single Point of Entry). Его основные задачи включают маршрутизацию запросов, агрегацию данных из различных сервисов (в перспективе), выполнение функций обратного прокси и трансляцию протоколов (Protocol Translation) из внешнего REST/JSON во внутренний gRPC/Protobuf. Использование шлюза позволяет скрыть детали реализации бэкенда от клиента.
2. **Invoices Service (Внутренний контур):** Является <<сервисом-источником истины>> для финансовых документов. Он инкапсулирует всю бизнес-логику: валидацию сумм (`amount`), генерацию идентификаторов и управление состоянием счетов. Сервис спроектирован так, чтобы быть максимально изолированным и независимым от способа отображения данных клиенту.

2.2 Стек протоколов и обоснование выбора

Для обеспечения баланса между доступностью и производительностью в системе реализован гибридный подход к коммуникациям:

- **REST (HTTP/1.1 + JSON):** Используется для взаимодействия <<клиент-шлюз>>. Выбор обусловлен широкой поддержкой протокола всеми современными языками программирования и браузерами. JSON обеспечивает легкость отладки и читаемость сообщений.
- **gRPC (HTTP/2 + Protobuf):** Используется для взаимодействия <<шлюз-сервис>>. В отличие от REST, gRPC использует бинарную сериализа-

цию Protobuf, что существенно (в 3-5 раз) уменьшает размер передаваемого трафика. Использование HTTP/2 позволяет мультиплексировать несколько запросов в одном TCP-соединении, что критично для высоконагруженных распределенных систем.

2.3 Модель управления данными и жизненный цикл

В текущей реализации используется оперативная память (in-memory) для хранения объектов `invoice`. Это решение позволяет минимизировать время отклика системы на этапе тестирования. Данные организованы в виде потокобезопасного словаря, где ключом выступает строковый ID.

Архитектура системы поддерживает горизонтальное масштабирование: при увеличении нагрузки можно запустить несколько экземпляров `invoices-svc-s1`. Однако, для корректной работы в таком режиме потребуется внедрение распределенной базы данных (например, PostgreSQL или Redis), что предусмотрено интерфейсом репозитория в коде.

2.4 Оркестрация и Service Discovery

Взаимодействие компонентов в динамической среде Docker обеспечивается встроенным механизмом Service Discovery. Шлюз не использует статические IP-адреса, а обращается к бэкенду по его логическому имени `invoices-svc-s13`. Внутренняя DNS-служба Docker разрешает это имя в актуальный IP-адрес контейнера, что позволяет прозрачно обновлять и перезапускать сервисы.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КОМПОНЕНТОВ

3.1 Реализация gRPC-контракта

Сердцем системы является файл `invoices.proto`, определяющий строгую структуру данных и доступные RPC-вызовы.

Listing 1: Структура контракта Invoices (`invoices.proto`)

```
1 syntax = "proto3";
2 package invoices.v1;
3
4 service InvoicesService {
5     rpc GetInvoice (GetInvoiceRequest) returns (GetInvoiceResponse)
6         ;
7     rpc CreateInvoice (CreateInvoiceRequest) returns (
8         CreateInvoiceResponse);
9     rpc ListInvoices (ListInvoicesRequest) returns (stream Invoice)
10        ;
11 }
12
13 message Invoice {
14     string id = 1;
15     string name = 2;
16     float amount = 3;
17 }
```

3.2 Разработка бэкенд-сервиса (Invoices Service)

Бэкенд реализован на языке Python с использованием библиотеки `grpcio`. Он содержит in-memory хранилище для оперативной работы с данными. Ключевой особенностью является использование генераторов для реализации метода `ListInvoices`, что обеспечивает эффективный стриминг данных клиенту.

Listing 2: Реализация создания и листинга счетов

```
1 class InvoicesServiceServicer(invoices_pb2_grpc.
2     InvoicesServiceServicer):
3     def CreateInvoice(self, request, context):
4         new_id = str(len(self._invoices) + 1)
```

```

4         invoice = invoices_pb2.Invoice(id=new_id, name=request.
           name, amount=request.amount)
5         self._invoices[new_id] = invoice
6         return invoices_pb2.CreateInvoiceResponse(invoice=invoice
           )
7
8     def ListInvoices(self, request, context):
9         for invoice in self._invoices.values():
10             yield invoice

```

3.3 Реализация REST-шлюза (API Gateway)

Шлюз разработан с использованием асинхронного фреймворка FastAPI. Его главная функция --- прием HTTP-запросов, преобразование их в gRPC-вызовы к внутреннему сервису `invoices-svc-s13` (обращение происходит по имени контейнера в Docker-сети) и возврат JSON-ответа.

Listing 3: REST-эндпоинт для создания счета

```

1 @app.post("/api/invoices", response_model=InvoiceSchema,
   status_code=201)
2 async def create_invoice(invoice: CreateInvoiceSchema):
3     client = get_grpc_client() #           : insecure_channel('
           invoices-svc-s13:8228')
4     try:
5         response = client.CreateInvoice(invoices_pb2.
           CreateInvoiceRequest(
6             name=invoice.name,
7             amount=invoice.amount
8         ))
9         i = response.invoice
10        return InvoiceSchema(id=i.id, name=i.name, amount=i.
           amount)
11    except Exception as e:
12        raise HTTPException(status_code=500, detail=str(e))

```

3.4 Система обработки исключительных ситуаций

В шлюзе реализован механизм перехвата ошибок. Например, если метод `GetInvoice` возвращает код `grpc.StatusCode.NOT_FOUND`, шлюз корректно

трансформирует его в ошибку HTTP 404, сохраняя семантику REST API для конечного потребителя.

4 ИНФРАСТРУКТУРА: КОНТЕЙНЕРИЗАЦИЯ И CI/CD

4.1 Контейнеризация сервисов (Docker)

Для обеспечения независимости от среды выполнения оба сервиса упакованы в Docker-контейнеры на базе минималистичного образа `python:3.11-slim`. В образах настраивается переменная окружения `PYTHONPATH` для корректного импорта сгенерированных gRPC-модулей. Оркестрация локального запуска осуществляется через `docker-compose.yml`.

Listing 4: Фрагмент `docker-compose.yml`

```
1 services:
2   invoices-svc-s13:
3     build: { context: ., dockerfile: invoices_svc.Dockerfile }
4     ports: ["8228:8228"]
5   gateway:
6     build: { context: ., dockerfile: gateway.Dockerfile }
7     ports: ["8000:8000"]
8     depends_on: [invoices-svc-s13]
```

4.2 Развертывание в кластере (Kubernetes)

Для развертывания в промышленных условиях подготовлен манифест `k8s.yml`. Он включает в себя ресурс `Deployment`, обеспечивающий запуск 2 реплик (`replicas: 2`) бэкенд-сервиса для повышения отказоустойчивости, и ресурс `Service` типа `ClusterIP` для внутрикластерной балансировки нагрузки на порт 8228.

4.3 Автоматизация CI/CD (GitHub Actions)

В рамках проекта внедрена практика непрерывной интеграции. Настроен пайплайн (`.github/workflows/week-17.yml`), который автоматически запускается при каждом `push`-запросе в главную ветку.

Пайплайн разворачивает виртуальную машину `Ubuntu`, устанавливает зависимости и запускает автоматические тесты `pytest`.

Listing 5: Шаг тестирования в GitHub Actions

```
1   - name: Run tests
2     run: |
3       STUDENT_ID=s13 GROUP=433 python3 -m pytest week-17/tests/
        test_week_17.py
```

Это гарантирует, что в главную ветку репозитория попадает только верифицированный и рабочий код, соответствующий архитектурным требованиям варианта s13.

ЗАКЛЮЧЕНИЕ

В результате выполнения работы была спроектирована и реализована распределенная система управления счетами `invoices-s13`. Основной акцент был сделан на использовании современных технологий межсервисного взаимодействия и обеспечении прозрачности API для внешних потребителей.

Были успешно решены задачи по проектированию gRPC-контракта, реализации шлюза на FastAPI и контейнеризации приложения. Полученные в ходе работы навыки проектирования микросервисной архитектуры являются базой для разработки масштабируемых промышленных приложений.